

1) Can we override a private method in Java?

- No, we cannot override a private method in Java.
- Because private methods are not visible to child class.
- So child class cannot access and override that method.

Example:

```
class Parent {  
    // Step 1: Private method  
    private void show() {  
        System.out.println("Hello Parent");  
        // this method is not visible outside this class  
    }  
}  
  
class Child extends Parent {  
    // Step 2: Same method name  
    void show() {  
        System.out.println("Hello Child");  
        // this is NOT overriding, it's a new method  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
  
        Child obj = new Child();  
        obj.show(); // calls Child method only  
  
    }  
}
```

Output:
Hello Child

2) How can we create a String object in Java?

- We can create String in two ways:
using string literal and using **new** keyword.
- Literal method stores string in String Constant Pool.
- **new** keyword creates a new object in heap memory.

Example:

```
public class Test {  
    public static void main(String[] args) {
```

```
        // Step 1: Using string literal
```

```
        String s1 = "Java"; // stored in String Constant Pool
```

```
        // Step 2: Using new keyword
```

```
        String s2 = new String("Java"); // creates new object in heap memory
```

```
        // Step 3: Compare both
```

```
        System.out.println(s1 == s2); // false → different memory locations
```

```
        System.out.println(s1.equals(s2)); // true → values are same
```

```
    }  
}
```

Output: false
true

3) What is String Constant Pool in Java?

- String Constant Pool is a special memory area inside heap.
- It stores only one copy of each string value to save memory.
- If same string is created again, it reuses existing one.

Example:

```
public class Test {  
    public static void main(String[] args) {
```

```
        // Step 1: Create strings using literals
```

```
        String s1 = "Java";
```

```
        String s2 = "Java";    // both refer to same object in String Pool
```

```
        // Step 2: Create string using new keyword
```

```
        String s3 = new String("Java"); // this creates new object in heap (not reused)
```

```
        // Step 3: Compare references
```

```
        System.out.println(s1 == s2);    // true → same reference from pool
```

```
        System.out.println(s1 == s3);    // false → different object in heap
```

```
    }  
}
```

Output: true
false

FOR MORE, MOST ASKED JAVA INTERVIEW QUES-ANS IN ONE PDF

DOWNLOAD PDF → [Click here to download PDF](#)

ONE SIMPLE ORGANIZED GUIDE

8 DAYS

QUICK

PREPARATION



DAYS	SUBJECT	
DAY 1	Core Java (Topic wise QA + Coding)	Startups & Big MNCs
DAY 2	Core Java (Most Important QA + Coding)	
DAY 3	DSA Coding Practice	
DAY 4	Advanced Java (Topic wise QA + Coding)	
DAY 5	Advanced Java (Most Important QA + Coding)	
DAY 6	TCS + Infosys + Wipro + HCL	
DAY 7	Cognizant + Capgemini + Accenture	
DAY 8	HR Round QA	